

Computer Architecture Lab Report

Spring 2025

Ali Zilouchi, Aydin Kazemi, Majid Sadeghinejad
810101560, 810101561, 810101459

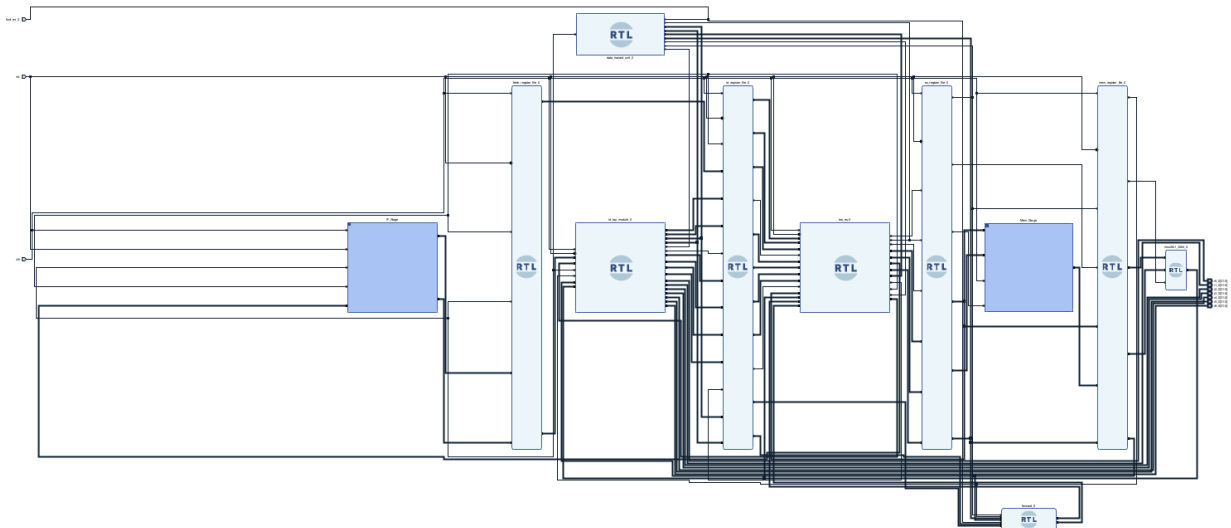
Introduction

In this project, we designed and implemented a five-stage pipelined ARM-style processor on an FPGA development board, following the standard IF–ID–EX–MEM–WB pipeline architecture. In the Instruction Fetch stage (IF), instructions are read from instruction memory using the program counter, which is then incremented or updated on branch. In the Instruction Decode stage (ID), register operands are read from the register file and immediates are sign-extended. The Execution stage (EX) performs arithmetic or logical operations via the ALU, taking into account forwarded data when available. The Memory stage (MEM) handles data loads and stores, while the Write-Back stage (WB) writes results back into the register file. This division of labor enables one instruction to be processed in each stage simultaneously, increasing throughput, although it introduces hazards that must be detected, stalled, or bypassed.

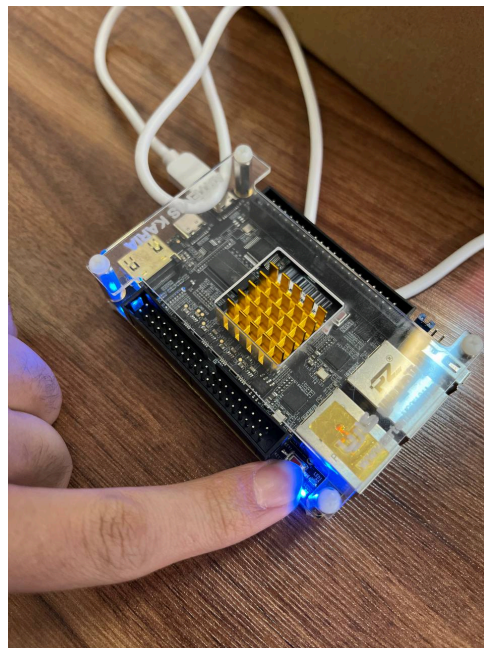
All design entry, simulation, synthesis, and implementation steps were carried out in Xilinx Vivado. Vivado provides an integrated environment for writing RTL in Verilog or VHDL, running behavioral and post-synthesis simulations, and performing timing-driven synthesis and place-and-route for Xilinx FPGAs. Key features we leveraged include the built-in simulator, the integrated logic analyzer for on-chip debugging (ILA), the resource utilization and timing reports, and the bitstream generation flow that programs the FPGA directly from the tool.

On the physical front panel of the development board, push-buttons and switches provide input signals that must be debounced before being used by the synchronous logic. We therefore implemented a Debouncer module that filters out the mechanical chatter of each push-button, producing a clean, single-transition digital pulse for each actuation. Without debouncing, spurious transitions could corrupt instruction fetch addresses or cause unintended control signals to fire, leading to erratic behavior in the pipeline.

The following picture shows our complete CPU design as a block diagram with all five pipeline stages, forwarding and hazard-detection units, and control signal paths. Below it is a photograph of the FPGA board running our processor, with the debounced push-buttons and LED outputs visible. These images document both the logical organization of our CPU and the hardware platform on which it is implemented.



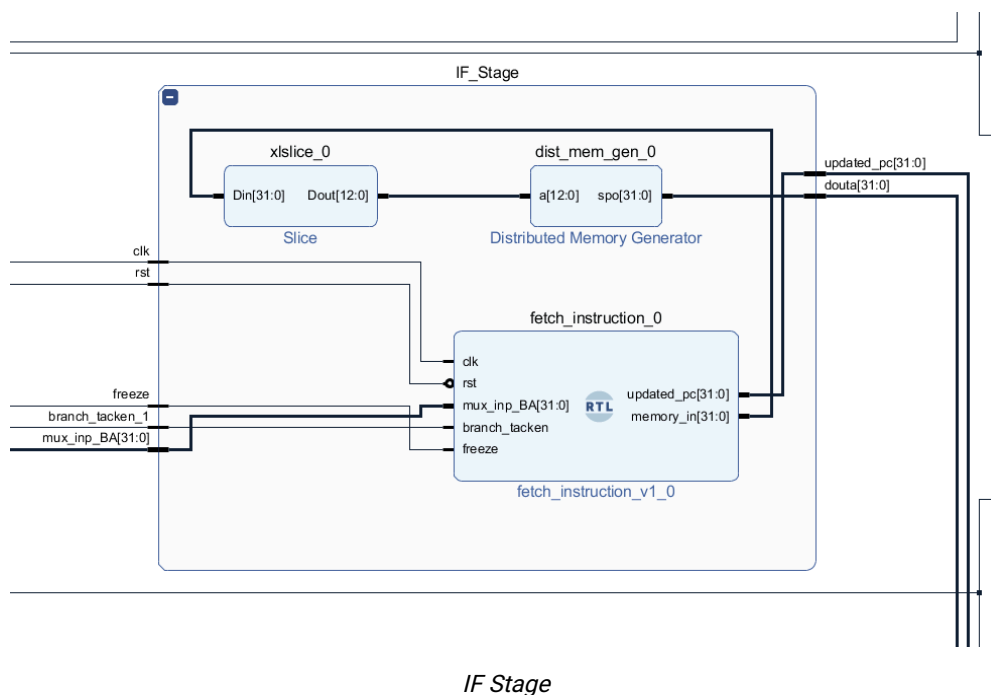
Full pipeline schema



Xilinx board used

Part One

The IF stage fetches instructions from memory using a program counter (PC) that increments sequentially or jumps on branches. The fetch_instruction module handles PC updates, while the fetch_register_file buffers the PC and instruction for the next stage. A distributed memory (8192 * 32) stores the program, initialized via a .coe file for simulation. Control signals (**freeze**, **flush**, **branch**) ensure correct pipeline flow during stalls or hazards. This stage ensures smooth instruction delivery to the decode unit.



IF Stage

Instruction Fetch

The Instruction Fetch (IF) module is the first stage of the pipeline ARM processor, responsible for fetching instructions from memory. It takes inputs such as the clock (**clk**), reset (`rst`), branch address (**mux_inp_BA**), and control signals (`branch_tackn`, `freeze`). The module outputs the updated program counter (`updated_pc`) and the memory address (`memory_in`) for instruction retrieval. A multiplexer (`mux_out`) selects between the next sequential address (`updated_pc`) and a branch target address (`mux_inp_BA`) based on the `branch_tackn` signal. The program counter (PC) increments by 1 each cycle unless a reset occurs or the pipeline is frozen (`freeze`). This module ensures correct instruction flow by handling branches and pipeline stalls efficiently. The output `memory_in` provides the address to the instruction memory, while `updated_pc` prepares the next instruction address for the pipeline.

```

module fetch_instruction (input wire clk, rst,
                        input wire [31:0] mux_inp_BA,
                        output wire [31:0] updated_pc, memory_in,
                        input wire branch_tacken, freeze);

    wire [31:0] mux_out;

    assign mux_out = ~branch_tacken ? updated_pc : mux_inp_BA;

    reg [31:0] PC;

    always @(posedge clk, posedge rst) begin
        if (rst) PC <= 0;
        else if (~freeze) PC <= mux_out;
    end

    assign updated_pc = PC + 1;

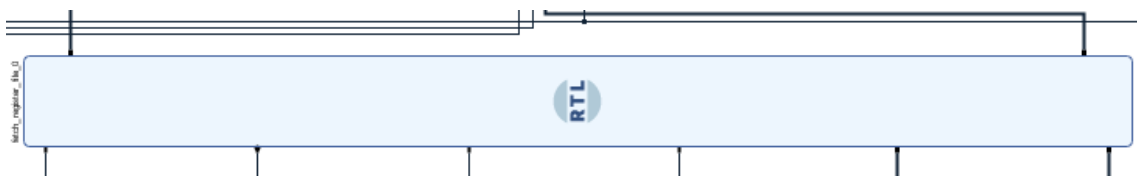
    assign memory_in = PC;

endmodule

```

Fetch Register File

The Fetch Register File module serves as an intermediate storage element between the Instruction Fetch (IF) and Instruction Decode (ID) stages in the pipeline. Its primary function is to hold the updated program counter value and the fetched instruction while accounting for pipeline control signals. The module operates on the positive clock edge and responds to three key control signals: reset (rst), pipeline flush (flush), and pipeline freeze (freeze). When reset is active, both the program counter and instruction registers are cleared to zero. If freeze is not active, the module checks the flush signal; when flush is asserted, the registers are cleared to handle branch mispredictions or hazards, otherwise, they store the incoming updated program counter and instruction values.



IF register file

```

module fetch_register_file (input wire clk,rst, input wire flush,freeze,
input wire [31:0] updated_pc,instruction,output reg [31:0] updated_pc_reg,instruction_reg);
    always @(posedge clk, posedge rst) begin
        if (rst) begin
            updated_pc_reg <= 0;
            instruction_reg <= 0;
        end
        else if (~freeze) begin
            if (flush) begin
                updated_pc_reg <= 0;
                instruction_reg <= 0;
            end
            else begin
                updated_pc_reg <= updated_pc;
                instruction_reg <= instruction;
            end
        end
    end
end
endmodule

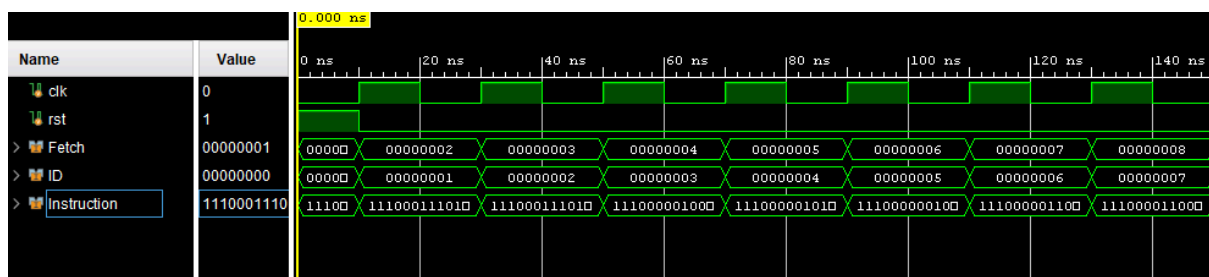
```

Instruction Memory

The instruction memory was implemented using Vivado's Distributed Memory Generator with 8K x 32-bit configuration, storing test programs loaded via a .coe file. This read-only memory provides low-latency access to instructions during fetch. The 8192-depth design balances capacity and resource efficiency while supporting 32-bit ARM instructions. Initialized with test code, it enabled smooth pipeline verification in simulation.

Simulation

As you can see in the picture below the pc is transferring between different stages and our testbench program loaded successfully:



Testbench

The testbench used for simulation is shown below. It is a minimal setup that only initializes the clock and reset signals and instantiates the top-level design. Please note that this is not

the testbench used for hardware verification, which included more output signals for observation and debugging during execution. That more detailed hardware testbench is not available here:

```
`timescale 1ns/1ns

module testbench1();
    reg clk = 0, rst = 0, fwd = 1;

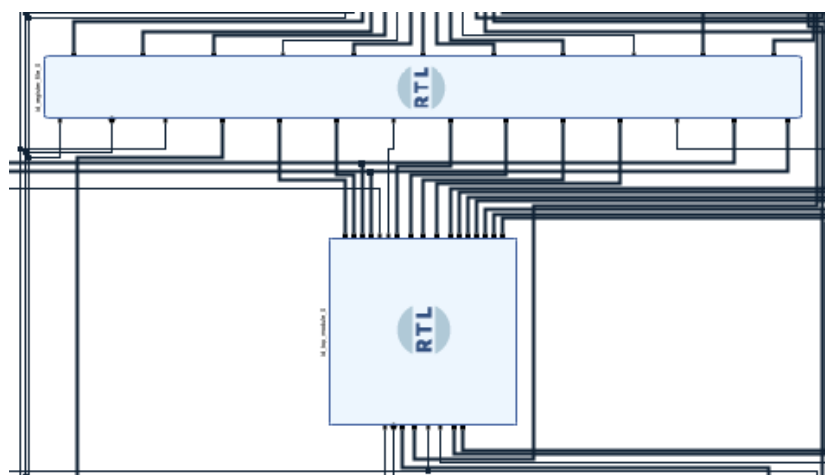
    CA_LAB1_wrapper uut (.clk(clk), .rst(rst));

    always #10 clk = ~clk;

    initial begin
        rst = 1;
        #10 rst = 0;
        #10000 $stop;
    end
endmodule
```

Part Two

The Instruction Decode (ID) stage decodes raw instructions into executable commands by extracting opcodes, registers, and immediate values. It reads operands from the register file and evaluates conditions using status flags (Z, C, N, V) to enable ARM's conditional execution. The control unit generates pipeline signals for ALU ops, memory access, and write-back, while hazard detection inserts stalls when needed. Immediate values and offsets are prepared for arithmetic or memory operations. Finally, a pipeline register buffers all decoded data (operands, controls, PC) for the Execute stage, supporting flushes on branches or hazards. This stage ensures correct instruction interpretation before execution.



ID Stage

Control Unit Module

The control unit decodes instruction opcodes and generates pipeline control signals for execution. It takes a 4-bit opcode, 2-bit mode, and status flag as inputs, producing a 9-bit control word output. The output signals include ALU operation codes, memory access controls, and write-back enables, with specific bit assignments for each function. A case statement maps instruction types to their corresponding control signals, handling data processing, memory, and branch operations. For arithmetic/logical instructions like ADD or AND, it sets the appropriate ALU operation code and write-back flag. Memory operations such as LDR activate memory read/write signals, while branches are detected when the mode bits equal 2'b10. The status flag propagates directly to the output, enabling conditional execution. The design efficiently encodes all necessary control signals while handling special cases like comparisons (CMP, TST) separately. Default cases ensure undefined instructions produce null control signals, maintaining pipeline stability.

```
module control_unit (input wire [3:0] opcode, input wire [1:0] mode, input wire s, output wire [8:0] cuo);
    // 8 7 6:3 2 1 0
    // s b exe memw memr wb
    assign cuo[8] = s;
    reg [7:0] temp;
    always @(*) begin
        case ({mode, opcode})
            6'b001101: temp = 8'b00001001; //MOV
            6'b001111: temp = 8'b01001001; //MVN
            6'b000100: temp = 8'b00010001; //ADD
            6'b000101: temp = 8'b00011001; //ADC
            6'b000010: temp = 8'b00100001; //SUB
            6'b000110: temp = 8'b00101001; //SBC
            6'b000000: temp = 8'b00110001; //AND
            6'b001100: temp = 8'b00111001; //ORR
            6'b000001: temp = 8'b01000001; //EOR
            6'b001010: temp = 8'b00100000; //CMP
            6'b001000: temp = 8'b00110000; //TST
            6'b010100: temp = s ? 8'b00010011 : 8'b00010100; //LDR
            // 6'b0101000: temp = 8'b00010100; //STR
            default: temp = 8'd000000;
        endcase
        temp = (mode==2'b10)? 8'b10000000 : temp; //BRANCH
    end
    assign cuo[7:0] = temp;
endmodule
```

Condition Check Module

The condition check module evaluates whether an instruction should execute based on the current processor status flags and the instruction's condition field. This implementation follows the ARM architecture's conditional execution model, where instructions only proceed if specific status flag conditions are met. The module takes four status flags as inputs: Zero (z), Carry (c), Negative (n), and oVerflow (v), along with a 4-bit condition code

(cond) from the instruction. It outputs a single bit (ccoo) indicating whether the instruction should execute (1) or be treated as a NOP (0). The implementation inverts the final result (assign ccoo = ~cco) to provide active-high execution enable. The case statement handles all 16 possible ARM condition codes: Conditions 0-7 test individual flags (EQ, NE, CS, CC, MI, PL, VS, VC), Conditions 8-9 test unsigned comparisons (HI, LS) using carry and zero flags, Conditions 10-13 test signed comparisons (GE, LT, GT, LE) using negative and overflow flag combinations, Condition 14 (AL) always executes, Condition 15 (NV) never executes. This module is crucial for implementing ARM's conditional execution feature, which improves code density and reduces branch penalties. The inverted output (ccoo) follows conventional active-high enable signaling, making it directly usable by pipeline control logic to gate instruction execution.

```
module condition_check (input wire [3:0] cond, input wire z, c, n, v, output wire ccoo);
    reg cco;
    always @(*) begin
        case (cond)
            4'd0: cco = z;
            4'd1: cco = ~z;
            4'd2: cco = c;
            4'd3: cco = ~c;
            4'd4: cco = n;
            4'd5: cco = ~n;
            4'd6: cco = v;
            4'd7: cco = ~v;
            4'd8: cco = c && ~z;
            4'd9: cco = ~(c && ~z);
            4'd10: cco = n == v;
            4'd11: cco = n != v;
            4'd12: cco = ~z && (n == v);
            4'd13: cco = ~(~z && (n == v));
            4'd14: cco = 1;
            4'd15: cco = 0;
            default: cco = 0;
        endcase
    end
    assign ccoo = ~cco;
endmodule
```

Register File Module

The register file serves as the processor's primary data storage, containing sixteen 32-bit registers that hold operands for arithmetic and memory operations. This synchronous design operates on the falling clock edge to avoid write-after-read hazards, with all writes completing in a single cycle when the write enable (we) signal is active. The module provides simultaneous read access to two registers (Rn and Rm) through dedicated output ports, while supporting write operations to any specified destination register (wDest). Seven

additional outputs expose specific registers (r0-r6) for debugging and status monitoring purposes. The design implements a synchronous reset that initializes all registers to their index values (register 0 contains 0, register 1 contains 1, etc.), ensuring predictable startup behavior. Key architectural features include the three-port design (two read, one write) that enables single-cycle read-modify-write operations, and the negative-edge clocking strategy that prevents read/write conflicts. The register file forms a critical component in the processor's data path, directly feeding the execution unit with operands while maintaining architectural state between instructions. Its efficient implementation supports the ARM instruction set's register-based operations while providing necessary visibility for debugging purposes.

```

module register_file_id (
    input wire clk, rst,
    input wire we,
    input wire [3:0] wDest,
    input wire [31:0] wValu,
    input wire [3:0] Rn,
    input wire [3:0] Rm,
    output wire [31:0] valRn,
    output wire [31:0] valRm,
    output wire [31:0] r0,r1,r2,r3,r4,r5,r6
);
    reg [31:0] registers [15:0];

    assign valRn = registers[Rn];
    assign valRm = registers[Rm];
    assign r0 = registers[0];
    assign r1 = registers[1];
    assign r2 = registers[2];
    assign r3 = registers[3];
    assign r4 = registers[4];
    assign r5 = registers[5];
    assign r6 = registers[6];

    integer i;

    always @(negedge clk, posedge rst) begin
        if (rst) begin
            for (i = 0; i < 16; i = i + 1) begin
                registers[i] <= i;
            end
        end
        else if (we) begin
            registers[wDest] <= wValu;
        end
    end
endmodule

```

ID Top Module

The ID top module integrates all components of the instruction decode stage, processing raw instructions and generating control signals for subsequent pipeline stages. It combines the condition checker, control unit, and register file to fully decode 32-bit ARM instructions while handling data hazards and conditional execution. The module first extracts status flags (Z, C, V, N) from the processor state and evaluates the instruction's condition field using the condition check submodule. The control unit decodes the opcode and mode bits to generate initial control signals (cuo), which are gated based on hazard detection and condition evaluation. The register file provides simultaneous read access to two operands, with source selection logic handling both register and immediate operand cases. Key outputs include decoded control signals (cuoo), operand values (val_rn, val_rm), addressing information (addr_rm, addr_rn), and immediate/offset fields. The design properly handles instruction variants through the two_src and imm flags, while preserving destination register information and constant fields for execution stage use. Special attention is given to hazard handling, where control signals are nullified when pipeline stalls are required.

```
module id_top_module(input wire clk, rst,
                    input wire [31:0] instr,
                    input wire [3:0] zcvn,
                    input wire hazard, w_en,
                    input wire [3:0] w_dest,
                    input wire [31:0] w_val,
                    output wire [31:0] val_rn, val_rm,
                    output wire [3:0] addr_rm, addr_rn,
                    output reg two_src,
                    output reg imm,
                    output reg [11:0] shift_operand,
                    output reg [8:0] cuoo,
                    output reg [23:0] signed_imm_24,
                    output reg [3:0] dest,
                    output wire [31:0] r0,r1,r2,r3,r4,r5,r6
                    );
    wire cco;
    wire [8:0] cuo;
    wire [3:0] rm_addr;

    wire z, c, n, v ;
    assign z = zcvn[3] ;
    assign c = zcvn[2] ;
    assign n = zcvn[0] ;
    assign v = zcvn[1] ;

    condition_check cc(instr[31:28], z, c, n, v, cco);

    control_unit cu(instr[24:21], instr[27:26], instr[20], cuo);

    register_file_id rf(clk, rst, w_en, w_dest, w_val, instr[19:16], rm_addr, val_rn, val_rm, r0,r1,r2,r3,r4,r5,r6);

    assign rm_addr = cuo[2]? instr[15:12] : instr[3:0];
    always@(*)
    begin
        cuoo = (hazard || cco)? 9'd0 : cuo;
        two_src = ~instr[25] || cuo[2];
        imm = instr[25];
        shift_operand = instr[11:0];
        signed_imm_24 = instr[23:0];
        dest = instr[15:12];
    end

    assign addr_rm = rm_addr ;
    assign addr_rn = instr[19:16] ;
endmodule
```

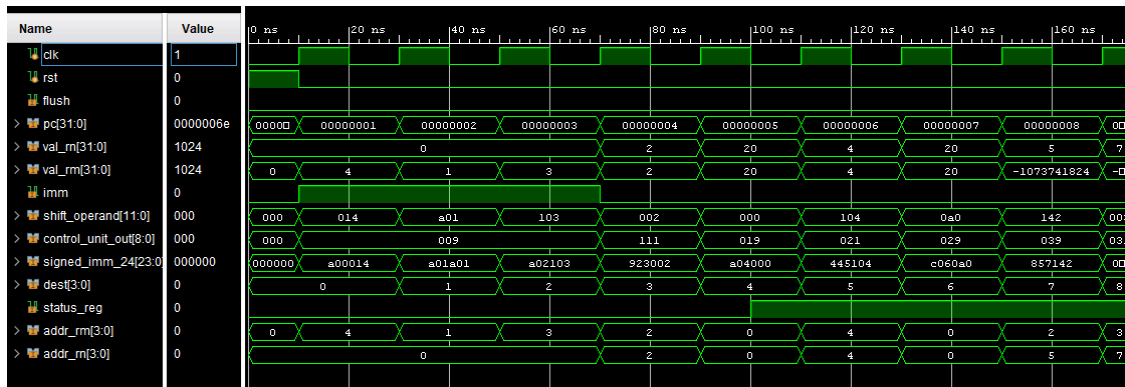
ID Register File (Pipeline Register)

The ID register file module acts as the pipeline buffer between the Instruction Decode and Execute stages, storing all critical instruction information for the next pipeline phase. This sequential element captures the complete decoded instruction state on every rising clock edge, including operand values, control signals, and addressing information, while providing essential pipeline control features. The design incorporates both reset and flush functionality to maintain pipeline integrity. A synchronous reset initializes all registers to zero, ensuring proper startup conditions, while the flush input clears the pipeline when handling branch mispredictions or hazards. Under normal operation, the register preserves the full instruction context, including the two source operand values and their original register addresses, the decoded control signals from the control unit, and various immediate value formats needed for execution. Key preserved elements include the program counter value for branch calculations, status register contents for conditional execution, and the destination register identifier for write-back operations. The register also maintains flags indicating immediate operand usage and different immediate value formats (12-bit shifted immediates and 24-bit signed offsets). By synchronizing all these elements to a single clock edge, the module ensures stable, coherent values throughout the execute stage's operation cycle.

```
module id_register_file (input wire clk,rst, input wire flush,
input wire [31:0] pc,val_rn,val_rm, input wire imm,
input wire [11:0] shift_operand, input wire [8:0] control_unit_out,
input wire [23:0] signed_imm_24, input wire [3:0] dest,input wire status_reg,
input wire [3:0] addr_rm , addr_rn,
output reg [31:0] pc_out,val_rn_out,val_rm_out, output reg imm_out,
output reg [11:0] shift_operand_out, output reg [8:0] control_unit_out_out,
output reg [23:0] signed_imm_24_out, output reg [3:0] dest_out,output reg status_reg_out,
output reg [3:0] addr_rm_out , addr_rn_out);
always @(posedge clk, posedge rst) begin
    if (rst) begin
        pc_out <= 0;
        val_rn_out <= 0;
        val_rm_out <= 0;
        imm_out <= 0;
        shift_operand_out <= 0;
        control_unit_out_out <= 0;
        signed_imm_24_out <= 0;
        dest_out <= 0;
        status_reg_out <= 0;
    end
    else if (flush) begin
        pc_out <= 0;
        val_rn_out <= 0;
        val_rm_out <= 0;
        imm_out <= 0;
        shift_operand_out <= 0;
        control_unit_out_out <= 0;
        signed_imm_24_out <= 0;
        dest_out <= 0;
        status_reg_out <= 0;
        addr_rm_out <= 0;
        addr_rn_out <= 0;
    end else begin
        pc_out <= pc;
        val_rn_out <= val_rn;
        val_rm_out <= val_rm;
        imm_out <= imm;
        shift_operand_out <= shift_operand;
        control_unit_out_out <= control_unit_out;
        signed_imm_24_out <= signed_imm_24;
        dest_out <= dest;
        status_reg_out <= status_reg;
        addr_rm_out <= addr_rm;
        addr_rn_out <= addr_rn;
    end
end
endmodule
```

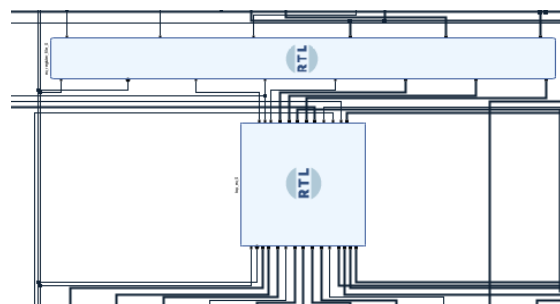
Simulation

As requested in this session description we simulate the status of ID register file signals for our test bench and as you can see everything is working well:



Part Three

In this session, we focused on implementing the execution (EXE) and write-back (WB) stages of our ARM processor pipeline. After completing the fetch and decode stages in earlier sessions, this part involved designing and integrating key components such as the ALU, Val2 Generator, branch address calculation logic, the status register, and the WB multiplexer. The ALU was responsible for performing arithmetic and logic operations based on the control signals and generating all relevant flags (N, Z, C, V). The Val2 Generator produced the appropriate second operand for the ALU, handling immediate values, offsets, and shift operations. We also implemented the logic for calculating branch target addresses and updating the condition flags accordingly. Finally, the WB stage was created using a simple multiplexer to select between memory output and the ALU result for writing back to the register file. Each component was tested through simulation, confirming correct functionality and alignment with the lab requirements.



ALU Module Explanation

This module performs arithmetic and logic operations based on a 4-bit **command** input. It supports instructions such as **MOV**, **ADD**, **ADC**, **SUB**, **SBC**, **AND**, **ORR**, **EOR**, and **MVN**. The result of the operation is assigned to **result**, and the status flags (**Z**, **C**, **V**, **N**) are packed into the 4-bit **zcvn** output. The **C** flag is used in **ADC** and **SBC** operations. The **Z** flag is set when the result is zero, **N** is set if the result is negative, and **V** is calculated for overflow in addition and subtraction. Here is the code for ALU module:

```
1  module ALU (
2      ...input wire [3:0] command, input wire [31:0] in1,in2,input wire C,
3      ...output reg [31:0] result, output reg [3:0] zcvn
4  );
5  always @(*) begin
6      ...zcvn = 4'b0000;
7      ...case (command)
8          ...4'b0001: begin // MOV
9              ...result = in2;
10             ...end
11             ...4'b0010: begin // ADD
12                 ...{zcvn[2], result} = in1 + in2;
13             ...end
14             ...4'b0011: begin // ADC
15                 ...{zcvn[2], result} = in1 + in2 + C;
16             ...end
17             ...4'b0100: begin // SUB
18                 ...{zcvn[2], result} = in1 - in2;
19             ...end
20             ...4'b0101: begin // SBC
21                 ...{zcvn[2], result} = in1 - in2 - !C;
22             ...end
23             ...4'b0110: begin // AND
24                 ...result = in1 & in2;
25             ...end
26             ...4'b0111: begin // ORR
27                 ...result = in1 | in2;
28             ...end
29             ...4'b1000: begin // EOR (XOR)
30                 ...result = in1 ^ in2;
31             ...end
32             ...4'b1001: begin // MVN
33                 ...result = ~in2;
34             ...end
35             ...default: begin
36                 ...result = 32'd0;
37             ...end
38         ...endcase
39     ...
40     ...// Z-flag (Zero)
41     ...zcvn[3] = (result == 32'd0);
42     ...
43     ...// N-flag (Negative)
44     ...zcvn[0] = result[31];
45     ...
46     ...// V-flag (Overflow) ... only for ADD, ADC, SUB, SBC
47     ...if (command == 4'b0010 || command == 4'b0011) // ADD, ADC
48         ...zcvn[1] = (in1[31] == in2[31]) && (result[31] != in1[31]);
49     ...else if (command == 4'b0100 || command == 4'b0101) // SUB, SBC
50         ...zcvn[1] = (in1[31] != in2[31]) && (result[31] != in1[31]);
51     ...end
52 endmodule
```

Val2 Generator Module Explanation

This module computes the second operand for the ALU based on the instruction type. It handles three cases:

- For **immediate instructions** ($I = 1$), it extracts an 8-bit immediate and rotates it as specified.
- For **memory instructions** ($Mem_in = 1$), it zero-extends the 12-bit offset.
- For **shifted register operands** ($I = 0$ and $Mem_in = 0$), it applies logical or arithmetic shifts (or rotate right) to val_Rm using the $shift_operand$.

The final computed value is sent to the ALU as the second input. Code for Val2Gen:

```
1 module Val2Generator(  
2     input [31:0] val_Rm,  
3     input I,  
4     input Mem_in,  
5     input [11:0] shift_operand,  
6     output reg [31:0] result  
7 );  
8  
9     reg [7:0] immediate_value;  
10    reg [31:0] rotated_value;  
11    reg [4:0] rotate_amount;  
12    reg [1:0] shift_type;  
13    reg [4:0] shift_imm;  
14  
15    always @(*) begin  
16        if (I == 1'b1) begin  
17            immediate_value = shift_operand[7:0];  
18            rotated_value = {24'b0, immediate_value};  
19            rotate_amount = 2 * shift_operand[11:8];  
20            result = (rotated_value >> rotate_amount) | (rotated_value << (32 - rotate_amount));  
21        end else if (Mem_in == 1'b1) begin  
22            result = {20'b0, shift_operand};  
23        end else if (I == 1'b0) begin  
24            if (shift_operand[4] == 1'b0) begin  
25                shift_type = shift_operand[6:5];  
26                shift_imm = shift_operand[11:7];  
27            end  
28            case (shift_type)  
29                2'b00: result = val_Rm << shift_imm;  
30                2'b01: result = val_Rm >> shift_imm;  
31                2'b10: result = $signed(val_Rm) >>> shift_imm;  
32                2'b11: result = (val_Rm >> shift_imm) | (val_Rm << (32 - shift_imm));  
33                default: result = val_Rm;  
34            endcase  
35            end else begin  
36                result = val_Rm;  
37            end  
38        end  
39    end  
40 endmodule
```

Status Register Module Explanation

This module stores the condition flags generated by the ALU. On the falling edge of the clock or upon reset, it updates its output flags (`zcvn_out`) only if the control signal `S` is active. The carry flag (`c_out`) is also extracted separately from the input. This allows the processor to preserve and reuse status information between instructions.

```
module status_register (
    input wire clk, rst,
    input wire S, input wire [3:0] zcvn,
    output reg [3:0] zcvn_out, output reg c_out
);
    always @(negedge clk, posedge rst) begin
        if (rst) begin
            zcvn_out <= 0;
            c_out <= 0;
        end
        else if (S) begin
            zcvn_out <= zcvn;
            c_out <= zcvn[2];
        end
    end
endmodule
```

EXE Stage Top Module Explanation

The `top_ex` module implements the Execution (EX) stage of the pipelined processor. It performs arithmetic and logical operations using the ALU, computes branch addresses, and passes relevant data and control signals to the next pipeline stages. The key operands for the ALU are `val_rn` and `val_rm`, with `val_rm` possibly being transformed by the `Val2Generator` to produce the final operand.

To support data forwarding and reduce stalls caused by hazards, two multiplexers controlled by `sel_src1` and `sel_src2` are used. `sel_src1` selects the first ALU operand from among the register value, a result forwarded from the memory stage, or the value from the write-back stage. Similarly, `sel_src2` chooses the source for `val_rm_out`, feeding the

Val2Generator. These multiplexers are essential for using the most recent register value even if it hasn't been written back yet.

In this session, these muxes were wired into the EX stage, but their control logic was not yet implemented. That logic will be added in Session 6 when the Forwarding Unit is built to produce **sel_src1** and **sel_src2** signals based on instruction dependencies.

Other components in the module include the ALU, which performs operations based on control inputs; the **Val2Generator**, which handles shift and immediate values; the status register, which updates condition flags if needed; and logic to calculate the branch address. The module also forwards necessary control signals for memory and write-back stages. This completes the core EX stage setup with support for future forwarding logic.

Here is the code for this part:

```
module top_ex (input wire clk,rst,
  ***input wire [31:0] pc,val_rn,val_rm, input wire imm,
  ***input wire [11:0] shift_operand, input wire [8:0] control_unit_out,
  ***input wire [23:0] signed_imm_24, input wire [3:0] dest,input wire status_reg_c,
  ***input wire [31:0] alu_res_mem, wb_value,
  ***input wire [1:0] sel_src1, sel_src2,

  ***output wire wb_en_out,mem_r_en_out,mem_w_en_out,
  ***output wire [31:0] alu_res_out,val_rm_out,branch_addr, output wire [3:0] dest_out,exe_dest,
  ***output wire carry_out, branch_tacken, exe_wb_en, output wire [3:0] zcvn_out
);

  ***assign val_rm_out = (sel_src2 == 2'b00) ? val_rm:
  ***                      (sel_src2 == 2'b01) ? alu_res_mem:
  ***                      (sel_src2 == 2'b10) ? wb_value:
  ***                      32'b0;

  ***wire [31:0] val2;
  ***Val2Generator v2g (.val_Rm(val_rm_out),.I(imm),
  ***.Mem_in(control_unit_out[2] | control_unit_out[1]),
  ***.shift_operand(shift_operand),.result(val2));

  ***wire [3:0] zcvn;

  ***wire [31:0] alu_inp_1;

  ***assign alu_inp_1 = (sel_src1 == 2'b00) ? val_rn:
  ***                      (sel_src1 == 2'b01) ? alu_res_mem:
  ***                      (sel_src1 == 2'b10) ? wb_value:
  ***                      32'b0;

  ***ALU alu(.command(control_unit_out[6:3]),.in1(alu_inp_1),.in2(val2),.C(status_reg_c),.result(alu_res_out),.zcvn(zcvn));
  ***status_register str(.clk(clk),.rst(rst),.S(control_unit_out[8]),.zcvn(zcvn),
  ***.zcvn_out(zcvn_out),.c_out(carry_out));

  ***assign branch_addr = {{8{signed_imm_24[23]}}, signed_imm_24} + pc;

  ***assign wb_en_out = control_unit_out[0];
  ***assign mem_r_en_out = control_unit_out[1];
  ***assign mem_w_en_out = control_unit_out[2];
  ***assign branch_tacken = control_unit_out[7];

  ***assign exe_dest = dest;
  ***assign exe_wb_en = control_unit_out[0];

  ***assign dest_out = dest;

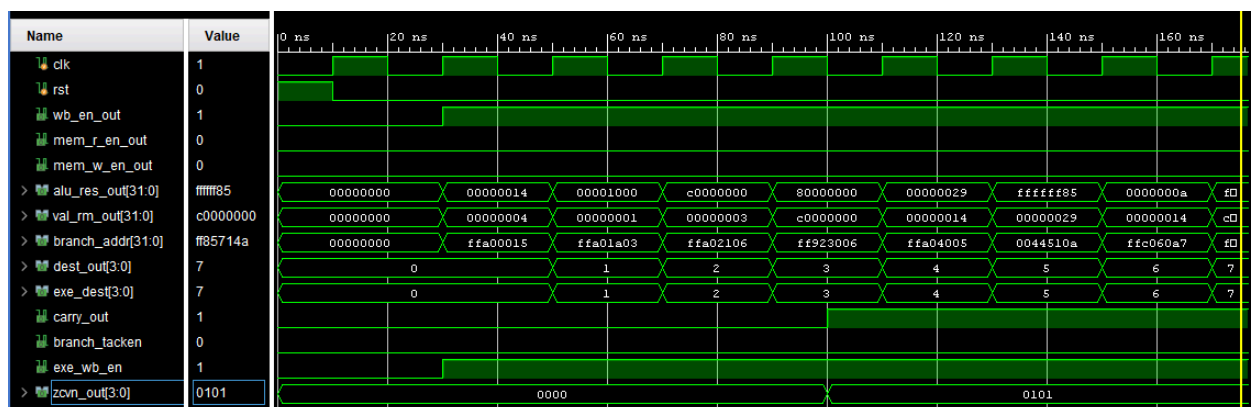
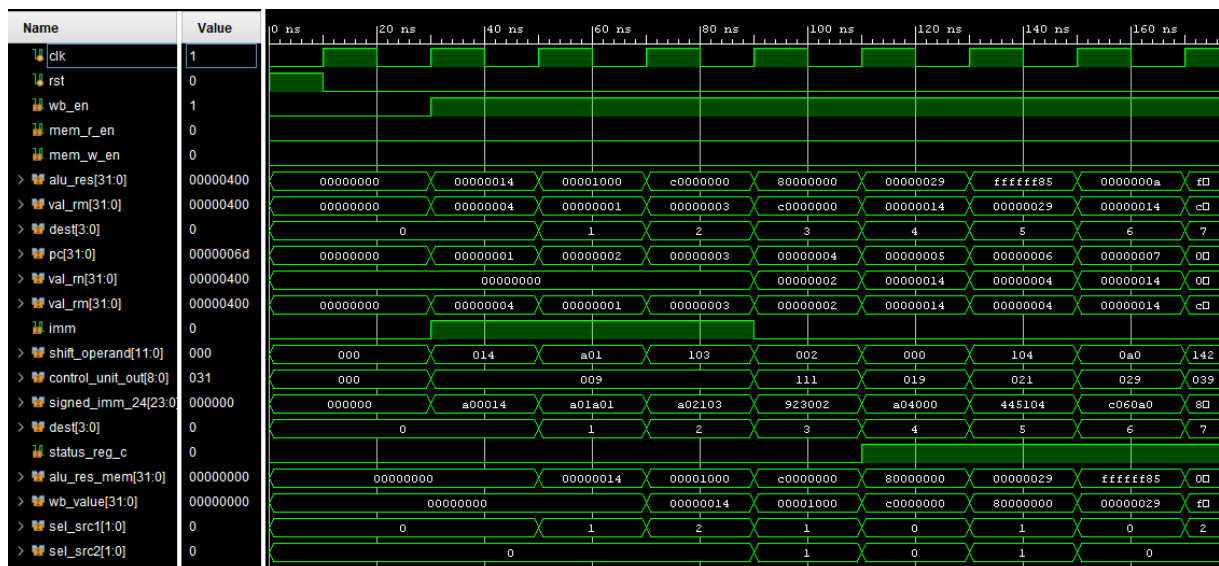
endmodule
```

The register file used in this stage is not shown here, as it remains the same as in previous sessions and does not require modification for the EXE or WB stages. After completing the

Verilog implementation of the modules discussed above, we proceeded with the hardware integration in Vivado. We followed all required steps including adding the design sources, applying constraints, synthesizing the design, and generating the bitstream. Once the Vivado setup was complete, we moved on to simulation using the provided testbench to verify the correct operation of the execution and write-back stages.

Simulation

We obtained the expected results from our simulation. Specifically, we were able to observe the correct execution of arithmetic and logic instructions, the proper behavior of the forwarding paths, and the correct update of status flags (**Z**, **C**, **V**, **N**). Instructions that did not involve data hazards executed as expected, confirming that the ALU and control signals functioned correctly. Below is a screenshot showing part of the simulation output:



Part Four

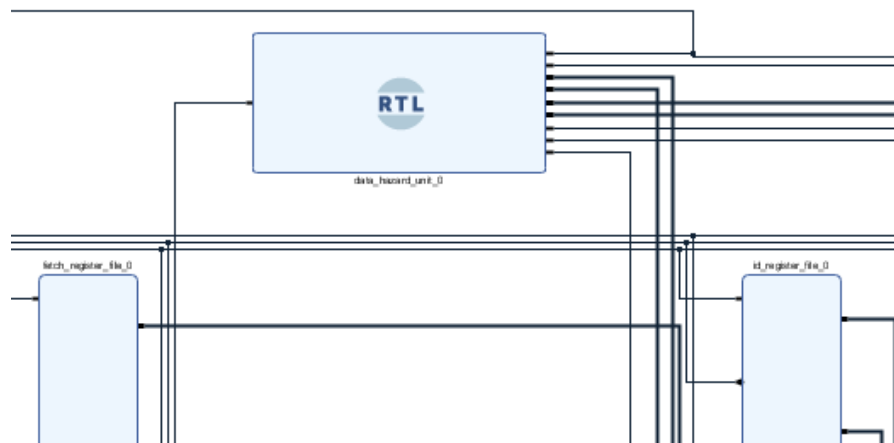
In this part of the lab, we implemented the final stages of the MIPS processor, including the memory access stage, write-back logic, and the hazard detection unit required for handling RAW hazards.

Hazard Detection Unit Explanation

This module takes into account whether the instruction has two source operands (`two_src`) and whether the instructions in later stages will write back to a register (`exe_wb_en`, `mem_wb_en`). If a hazard is detected—meaning the current instruction depends on a result that hasn't been written back yet—the module raises the `hazard` signal to stall the pipeline. The stall may last one or two cycles depending on which stage the hazard originates from.

In addition, the module supports integration with the forwarding logic planned for the next session. If `fwd_en` is active (i.e., forwarding is enabled), the hazard detection logic is adjusted. In this case, only load-use hazards (where an instruction depends on a value being loaded from memory) will cause a stall, and even then, only for a single cycle. This allows forwarding to handle most other dependency cases without needing to pause the pipeline. Overall, this module plays a key role in maintaining correct execution by managing when to apply stalls and when forwarding can safely take over. Here is the code for this part:

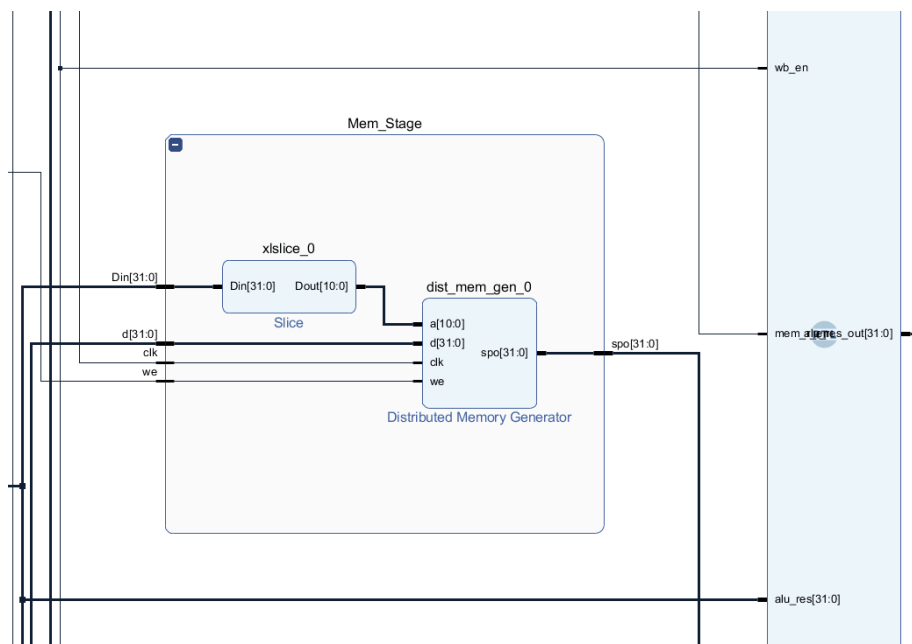
```
module data_hazard_unit (
  input wire two_src, mem_wb_en, exe_wb_en,
  input wire [3:0] mem_dest, exe_dest, rn, rm,
  input wire exe_mem_r_en, fwd_en,
  output wire hazard
);
  assign hazard = (~fwd_en)?
    ((rn == mem_dest) && mem_wb_en) ||
    ((rn == exe_dest) && exe_wb_en) ||
    (two_src && (rm == mem_dest) && mem_wb_en) ||
    (two_src && (rm == exe_dest) && exe_wb_en):
    (exe_mem_r_en)?
    (rn == exe_dest) ||
    (two_src && (rm == exe_dest)) :
    1'b0;
endmodule
```



Hazard Detection Module

Memory Explanation

The memory was implemented using a Vivado block RAM with configuration to support **combinational read** and **synchronous write**. Write operations occur on the rising edge of the clock when **MEM_W_EN** is high, and reads are performed immediately when **MEM_R_EN** is asserted. Address alignment and offset handling (subtracting 1224 and dividing by 4) were applied as required.



MEM Stage

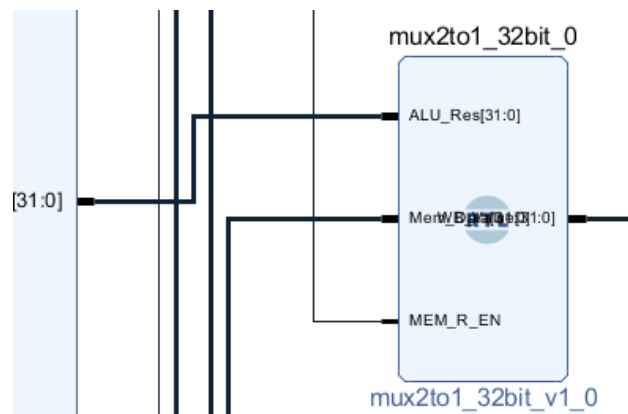
Write back explanation

This module selects between the ALU result and the memory output based on the **MEM_R_EN** control signal. If reading from memory, **Mem_Data** is selected; otherwise, the ALU result is forwarded. This value is written back to the register file.

Here is the code for this part:

```
module mux2to1_32bit(  
    input wire [31:0] ALU_Res,  
    input wire [31:0] Mem_Data,  
    input wire MEM_R_EN,  
    output wire [31:0] WB_Value  
);  
  
    assign WB_Value = MEM_R_EN ? Mem_Data : ALU_Res;  
  
endmodule
```

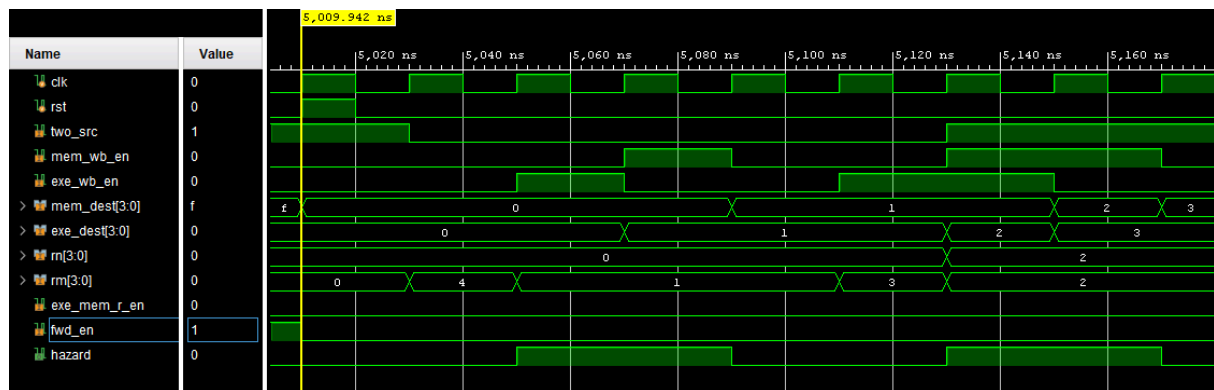
The register file used in this section is identical to the one implemented in previous stages, so we have omitted its code here. All modules, including the register file, were properly instantiated and connected in the Vivado project's top module, and the complete design was successfully tested.



Simulation Results

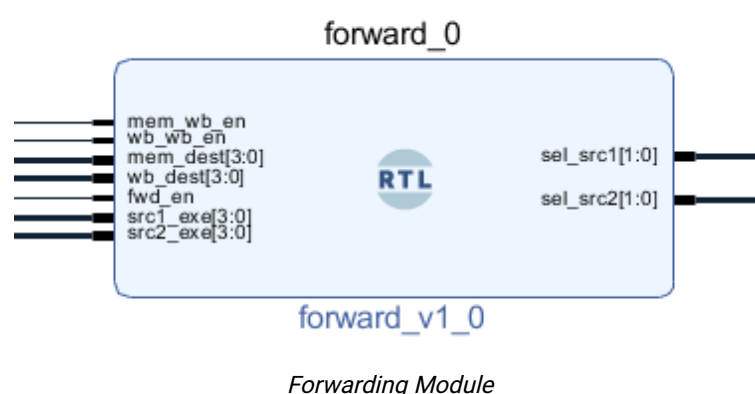
We completed all necessary steps in Vivado for simulation setup and execution. The same testbench used in the previous part was reused here for both behavioral and post-synthesis simulations to avoid redundancy. Therefore, we do not include or explain the testbench again.

According to the lab instructions, this section aimed to implement memory with proper read/write behavior, add the hazard detection unit for RAW hazards, and test the complete processor. The simulation results confirmed that the processor performed as expected, memory operations worked correctly, and RAW hazards were properly detected and handled:



Part Five

In this session, we implemented the forwarding technique in a pipelined ARM processor to resolve data hazards and reduce unnecessary stalls. We modified the processor's architecture by adding a forwarding unit and necessary control signals, then verified the correctness through simulation and evaluated the performance improvement compared to the non-forwarding version.



Hazard Detection Changes

This was already explained in the previous session's report, but to repeat: we modified the hazard unit to operate differently when forwarding is enabled. In this mode, it only detects

hazards if the next instruction is a load (LD), which causes the pipeline to stall for one cycle. The code for this change was included in the last session's report.

Forward Detection Module

This module receives as input the destination registers and write-back enable signals from both the MEM and WB stages (`mem_dest`, `wb_dest`, `mem_wb_en`, `wb_wb_en`), as well as the source registers of the instruction currently in the EXE stage (`src1_exe`, `src2_exe`). An additional control input `fwd_en` enables or disables forwarding functionality. Based on these inputs, the module produces two 2-bit control signals, `sel_src1` and `sel_src2`, which are used to guide multiplexers in the EX stage to choose the correct operand values.

When forwarding is disabled (`fwd_en` is low), both selection signals are set to 2'b00, indicating that operands should be taken directly from the register file. When enabled, the module checks whether either source operand matches the destination register of an instruction in the MEM or WB stage and whether that instruction will write back a result. If a match is found and the corresponding write-back enable signal is active, the corresponding select signal is updated to choose the forwarded value instead. This logic is essential for bypassing the register file and providing the most recent data directly to the ALU, thereby reducing unnecessary stalls and improving overall pipeline efficiency.

Here is the code for this module:

```
module forward (
    ... input wire mem_wb_en, wb_wb_en,
    ... input wire[3:0] mem_dest, wb_dest,
    ... input wire fwd_en,
    ... input wire[3:0] src1_exe, src2_exe,
    ... output wire[1:0] sel_src1, sel_src2
);
    ... assign sel_src1 = (~fwd_en)? 2'b00:
    ... ((src1_exe == mem_dest) && mem_wb_en)? 2'b01:
    ... ((src1_exe == wb_dest) && wb_wb_en)? 2'b10:
    ... 2'b00;
    ... assign sel_src2 = (~fwd_en)? 2'b00:
    ... ((src2_exe == mem_dest) && mem_wb_en)? 2'b01:
    ... ((src2_exe == wb_dest) && wb_wb_en)? 2'b10:
    ... 2'b00;
endmodule
```

Modified Top EXE Module

As it was already mentioned, two multiplexers were added to allow the forwarding unit to select which inputs are fed into the ALU. Additionally, new ports—specifically `sel_src1` and `sel_src2`—were introduced so that the internal ALUs can operate based on the control signals generated by the forwarding unit. A more detailed explanation of this module was provided in the EXE top module section of the previous session's report, so we avoid repeating them.

Modified ARM Design

Along with the changes mentioned in the previous section, we added the forwarding unit itself. We also modified the hazard unit—both of which were explained earlier. Additionally, `src1` and `src2` signals from the decode phase were added to the decode-to-RF stage, allowing us to pass them to the forwarding module so it can determine which inputs should be selected for the ALU.

Part Six

Simulation Results

Here as you can see, testbench has little changes from the previous parts, and the reason is that we wanted to compare the runtime of each of the modes of our cpu:

```
`timescale 1ns/1ns

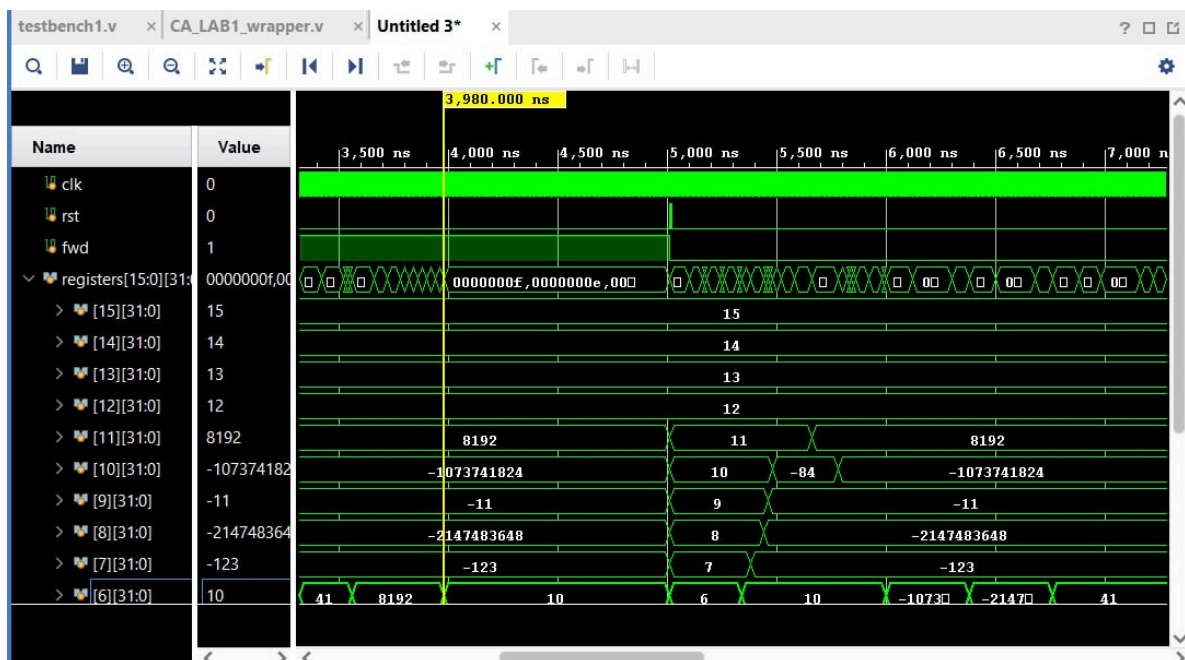
module testbench1();
    reg clk = 0, rst = 0, fwd = 1;

    CA_LAB1_wrapper uut (.clk(clk), .rst(rst), .fwd_en_0(fwd));

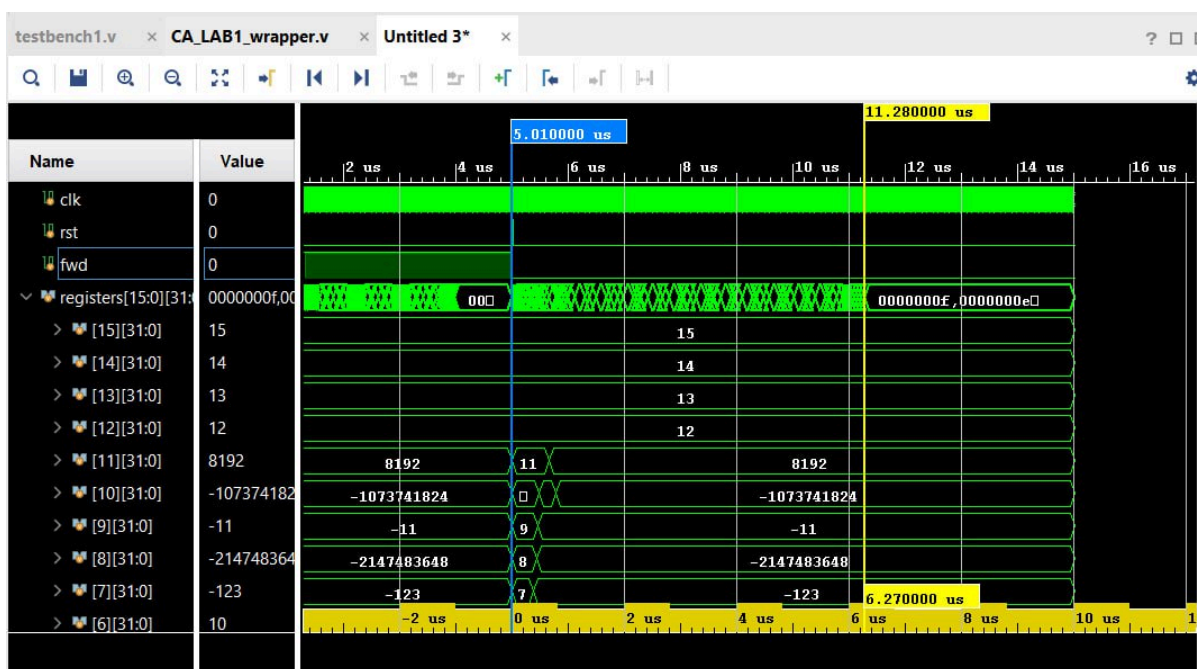
    always #10 clk = ~clk;

    initial begin
        rst = 1;
        fwd = 1;
        #10 rst = 0;
        #5000;
        fwd = 0;
        rst = 1;
        #10 rst = 0;
        #10000 $stop;
    end
endmodule
```

With that said, after setting up Vivado, we ran the simulation, and here is the results for both with and without forwarding enabled:



Simulation with Forwarding enabled

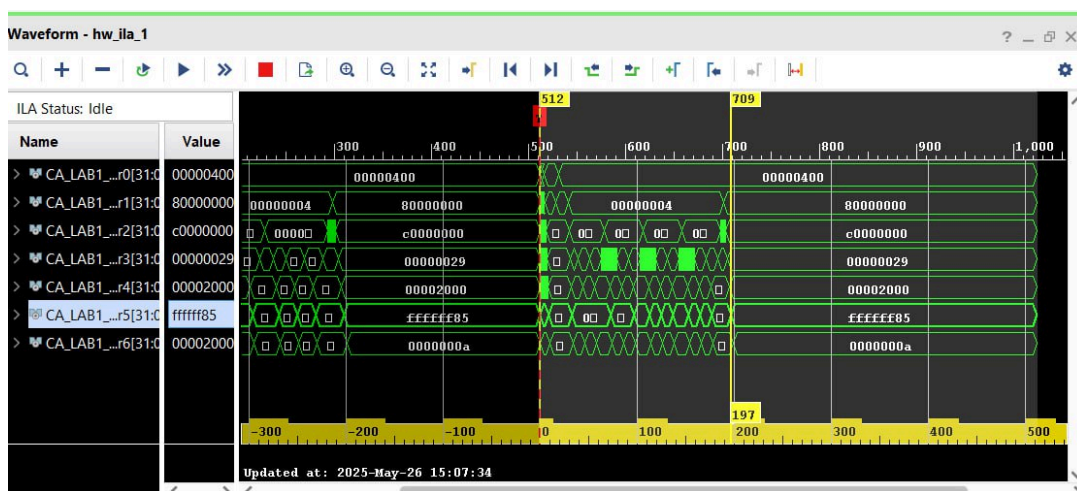


Simulation without Forwarding enabled (blue mark is the starting point)

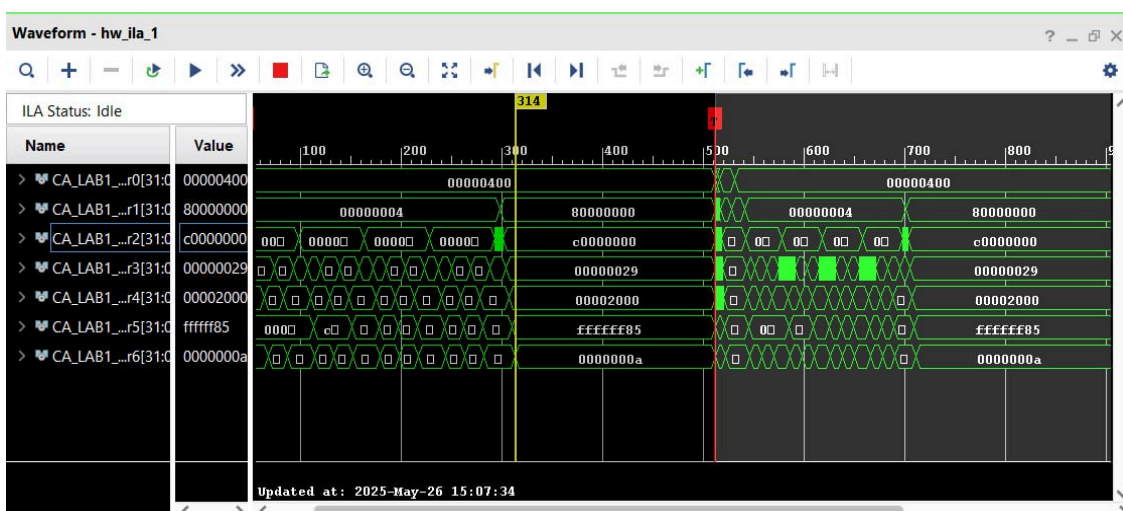
As shown here, the simulation with forwarding enabled completed in 3980 ns, while the one without forwarding took 6280 ns. Given a 20 ns clock cycle, this results in an approximate speedup of 1.58 $\times$ when forwarding is used.

Hardware Simulation Results

To compare the behavior of the processor with and without forwarding in hardware, we set up a split-screen environment using two simulation instances running side by side. This allowed us to observe and compare the output signals in real time as the same program was executed under both conditions. Since enabling forwarding required setting the `fwd_en` signal to `1`, and it was mapped to a physical button on the board, we created a workaround to avoid manually holding the button during the test. To achieve this, we modified the constraint file and design to use a virtual signal or switch, effectively simulating a "button always pressed" condition. This ensured consistent behavior during the forwarding-enabled simulation, improving accuracy and convenience. Here are the hardware simulation results:



Hardware Simulation with Forwarding enabled (197 timestamps)



Hardware Simulation without Forwarding enabled (314 timestamps)

Comparison

Here, as you can see, we get roughly the same result of **1.59× speedup** (314/197, the minor difference could be due to measurement errors), which confirms our simulation results and highlights how significant the impact of enabling forwarding is.

CPI

Hazard: 314/47 = 6.68

Forwarding: 199/47 = 4.23

Cycle count

Hazard: 314

Forwarding: 199

Hardware Resources (LUT + FF)

Hazard: 8340

Forwarding: 8890

Performance per Cost ($1e6/(\text{Resource} \times \text{CPI})$)

Hazard: 17.9

Forwarding: 26.6